LAMBDA: Landmark Alignment and Mixture-Based Domain Adaptation

Thibaut Martinet[1] and Guillaume Metzler[2]

[1] Université d'Orléans, LIFO EA 4022, Orléans 45067, France
[2] Université de Lyon, Lyon 2, ERIC UR 3083, Bron 69676, France

thibaut.martinet@univ-orleans.fr
guillaume.metzler@univ-lyon2.fr

Abstract

In this work, we propose a new unsupervised way to select landmarks and perform source-target matching, in the context of domain adaptation. More precisely, our contribution proposes (i) to select landmarks using Gaussian mixture models on the source and target sets and (ii) to match these landmarks using an algorithm from combinatorial. The proposed method is independent of the choice of the classifier and of the task at end. The interest of this approach is shown with empirical results obtained on synthetic data to illustrate the problems we are able to solve.

Keywords

Unsupervised Domain Adaptation, Mixture Model, Landmarks, Assignment Problem, Hungarian Algorithm.

1 Introduction

The most common tasks in Machine Learning make the assumption that the data used to learn and test the model follow the same distribution. In practice, this assumption is usually wrong [6] and this results in a drop of the performance of the model on the test data. We are then interested in a way of transferring and adapting the knowledge from the so-called *source* data to a new set of *target* data, having a different distribution from the source data. This is called *Transfer Learning* and more precisely *Domain Adaptation*[22].
Domain Adaptation has applications in many fields such as natural language processing [21] with applications in machine translation [5], or in image classification with approaches based on deep learning [16, 26] from a supervised or unsupervised point of view.
The present work is placed in an unsupervised domain adaptation context where only the source data are labeled. The objective is then to find a good transformation of the data so that the source model can perfectly fit the distribution of the target data without any knowledge on this domain other than the distribution of the features.
There are two main approaches to solving this task. One is to align, *i.e.* a reweighting mapping, the data from the source (or target) distribution with the target (or source) data [15, 24]. The other, more common, is to find a latent space in which the source and target data will be projected by minimizing the distance between the two domains in this space [1, 18, 25].

The present work is in another category and consists in learning a different representation for the source data and then the target data using landmarks. We will then try to match the features of each representation in order to make them coincide . Although a large part of the state of the art on domain adaptation focuses on deep approaches to solve this problem, we choose to approach this problem using statistical tools.
The contributions are the following: (i) landmarks are learned using Gaussian Mixture Models (GMM) [8] on the source and target sets separately. They are then used to learn a new representation of the source (resp. target) data using the landmarks learned on the source (resp. target) set. Secondly, (ii) we match these landmarks using an algorithm from combinatorial, the hungarian algorithm [17], in such a way that the model learned on the source data can be used on the training data based on different but coinciding features.

The remaining of the paper is organized as follows. Section 2 presents a state of the art on unsupervised domain adaptation. Section 3 presents the tools used to develop our approach, as well as our Landmark Alignment and Mixture-Based Domain Adaptation (LAMBDA) algorithm. Section 4 is dedicated to experiments on simulated data and we conclude in Section 5.

2 A Review on Domain Adaptation

Domain adaptation methods generally try to reconcile the distributions of the source and target domains by evaluating the distance between these two domains, for example with the *Maximum Mean Discrepancy* (MMD) [14] or the *Wasserstein* [19] distance to name but two (see [22] for a more complete list of divergence measures between domains). The main difficulty is that these notions of distances only use the marginal distributions of both source and target data (i.e. the distribution of the source and target features) because we do not have access the labels on the

target domain in Unsupervised Domain Adaptation.

However, the methods based on these notions of distances try to learn transformations of the source data to project them on the target data in a linear way (as with metric learning techniques [10, 27]) or in a non-linear way with approaches based on deep learning [25], but the latter will not be developed here. They also seek to reduce this distance by plunging the source and target data into a common latent space in which the marginal distributions (i.e. the plunging of the source and target feature space) is as close as possible [10]. This common latent space can usually be learned jointly with the classifier. This point of view is also adopted in the PAC-Bayesian Theory [12]. More precisely, the authors propose approaches that seek to minimize a bound on the risk in generalization on the target set that depends on the distance between the two domains [11, 13]. This distance, called *domain disagreement*, involves different voters defined with the help of kernel methods, thus projecting the data in a certain space in which one seeks to minimize both a risk on a source set and the distance between the source and target data. These approaches are interesting because they provide guarantees on the performance of the obtained classifier. On the other hand, many of the terms involved in these bounds can not be computed in practice, making the results only partially reliable or depending on several so called hyper-parameters.

A last proposed approach consists in carrying out the alignment process by not working directly on the original features, but rather on transformed features [1, 7], through the use of landmarks [1]. Landmarks are mainly used to approximate kernel methods by selecting or learning the most important point for the task at end, thus reducing the complexity of learning such models. If the use of landmarks remains limited in a domain adaptation context, they allow to generate a new data representation for which the task of alignment between domains will be simplified [1]. New features on the source and target data are computed according to the similarities (distance, or Gaussian similarity) of the points to these so-called landmarks [1]. A linear transformation of the data is then learned to align the obtained distributions with this new representation. One can for example select landmarks with a K-means or by analyzing the distribution of the data around potential candidates via the use of Gaussian similarities. This method is however very expensive because it requires to compute multiple distances. Moreover, the choice of landmarks is made on the distribution of similarities and not on the original features. This leads to a loss of information when choosing the landmarks.

Our method, presented in the next section, proposes to select landmarks based on the basic features and a way to align the distributions by matching the source and target features, thus freeing itself from the notions of distances usually used.

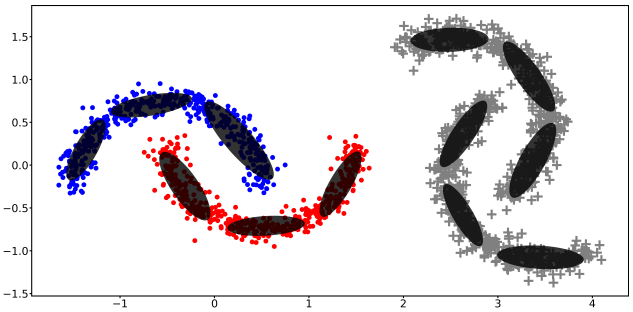


Figure 1: Example of a source (dots) and target (plus) moons dataset with a rotation and a translation between them, covered by the GMMs clusters. The different ellipsoids represent the clusters that are learned using the GMMs and the EM algorithms[1], *i.e.* how the clusters can be represented according to the learned mean vectors and covariance matrices.

3 A Statistical and Combinatorial Approach

In this section, we present **LAMBDA**, our novel approach to domain adaptation that leverages mixture models and the Hungarian algorithm, an algorithm derived from Combinatorial. We talk about two versions of the model, namely **LAMBDA1** and **LAMBDA2**, which differ in their landmarks alignment algorithm. And **LAMBDA** (without number) is about the mechanisms common to both versions.

Setting: The input space is denoted $\mathbf{X}$ and is a d-dimensional space, *i.e.* $\mathbf{X} \subset \mathbb{R}^d$, and the output space is denoted Y (it can represent the set of labels for either binary or multi-class classification). We also define $\mathcal{D}_S$ and $\mathcal{D}_T$ the two distributions over the source domain $\mathbf{X}_S \times Y$ and the target domain $\mathbf{X}_T$. We do not have access to the labels of the target domain, thus we are dealing with Unsupervised Domain Adaptation. Our goal is to learn an algorithm able to solve this task that consists, given a m_s-source sample $S_S = \{(\mathbf{x}_i^s, y_i)\}_{i=1}^{m_s}$ drawn *i.i.d.* from $\mathcal{D}_S$ and a m_t-target sample $S_T = \{(\mathbf{x}_i^t)\}_{i=1}^{m_t}$ drawn *i.i.d.* from $\mathcal{D}_T$, in inferring from S_S to S_T.

3.1 The Early Stages of LAMBDA

The **LAMBDA** algorithm works by placing the same number of landmarks into each distribution, and then aligning the source and target landmarks to indirectly align the two distributions, to allow a normal classifier to analyse their respective data as if it came from a single distribution, as shown in the flowchart in Figure 2.

Gaussian Mixture Model. GMM [8] is a probabilistic clustering model which estimates the probability distribution of the data by decomposing it into K normal distributions (or K gaussian clusters), as can be seen on the

[1] We use the mixture.GaussianMixture function in sklearn to learn the mixture models and represent the ellipses.

Figure 1. GMM estimates the parameters of the gaussian clusters, *i.e.* their means and covariances, using the EM (Expectation-Maximization) algorithm, which maximise the completed log-likelihood of the estimated distribution on the data:

$$\ell(\mathbf{x};\theta) = \sum_{k=1}^{K}\sum_{i=1}^{n} \tilde{z}_{ik}\left(\ln(p_k) - \frac{d}{2}\ln(2\pi) - \frac{1}{2}\ln\left(|\mathbf{\Sigma}_k|\right) - \frac{1}{2}(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top \mathbf{\Sigma}_k^{-1}(\mathbf{x}_i - \boldsymbol{\mu}_k)\right)$$

where $\forall i = 1,\dots,n$, $\mathbf{x}_i \in \mathbb{R}^d$ are the observed data, $\boldsymbol{\theta}$ are the parameters of the clusters ($\forall k = 1,\dots,K$, $\theta_k = (p_k, \boldsymbol{\mu}_k, \mathbf{\Sigma}_k)$), $\tilde{z}_{ik}$ is an indicator function which is equal to 1 if $\mathbf{x}_i$ belongs to the cluster k and 0 otherwise, p_k, $\boldsymbol{\mu}_k$ and $\mathbf{\Sigma}_k$ are respectively the proportion, the mean and the co-variance of the cluster k, and $\cdot^\top$ is used to denote the transpose.

GMM is similar to the more classical clustering model K-means, in that both work by successive iterations, where an iteration will move mean-points among the data relative to the previous iteration's estimation on the data. However, the K-means mean-points only influence the data uniformly on each dimension.
Actually, K-means could be expressed as a special case of GMM where the covariance matrices of its clusters are the same scalar matrix. There is even a parsimonious version of GMM [4] where the covariance matrices are fixed to the same scalar matrix (which simplifies its learning, in particular the M step of the EM algorithm), and the proportions are assumed to be equal. A variant of the EM algorithm can also be expressed as a classification algorithm (CEM [3]), where each example is assigned to the component of the mixture to which the example is most likely to belong. Using this CEM variant, and using the parsimonious version of GMM described above, the resulting model is strictly equivalent to K-means.
All this to say that GMM is a generalization of K-means, which will usually not only optimise the means of its clusters, but also their proportions and their covariance matrices, which makes it more accurate in estimating the data distribution.

Finding the best landmarks distributions. To determine the position of the landmarks, **LAMBDA** uses GMMs to estimate K gaussian clusters on both source and target distributions, K being a hyper-parameter of the model, and then considers the means of the estimated clusters as the landmarks.
But since the EM algorithm is a converging algorithm based on estimations, it will not always reach the global maximum, but will usually rather fall into local ones. In practice, to get rid of this problem, the EM algorithm is often run a large number of times from different initial values, to increase the chances of reaching an optimum, *i.e.* a better estimation of the data.

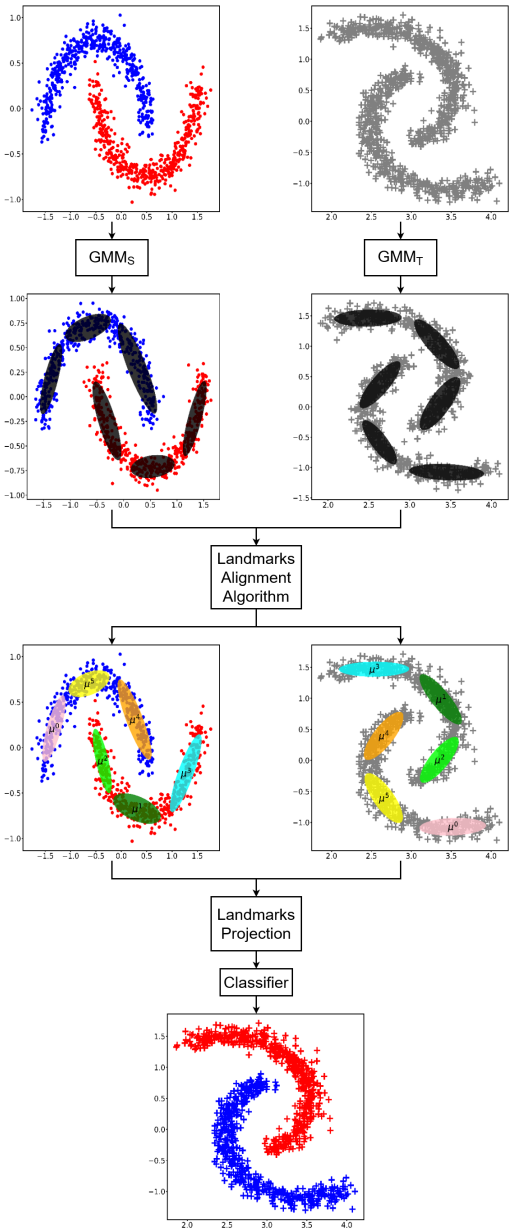


Figure 2: Flowchart of **LAMBDA** with its three steps: (i) a GMM is applied to find the landmarks, (ii) the landmarks source and target are then aligned or paired and (iii) a classifier is trained.

In order to compare two GMM estimations and select the best one, we can use the BIC (Bayesian Information Criterion [23]), a criterion that indicates which estimation is more likely according to the data.
For this purpose, **LAMBDA** fits a certain number (10 by default) of GMMs, and selects the one that maximises the BIC in each distribution independently. Thus, the landmarks are more likely to be placed in the densest areas of the data distributions, since they are means of estimated gaussians.

Landmarks alignment of LAMBDA1. Since the landmarks are expected to be similarly placed on the corre-

sponding dense areas of each distribution, the respective landmarks spaces, in which the source and target data are projected, are expected to coincide as well. So **LAMBDA** expresses each instance $\mathbf{x}$ by the distances to the K landmarks of its distribution, resulting in a K-dimensional vector.
Thus, the assumed coincidence allows a classic (*i.e.* a non-domain adaptation) classifier to handle S_S and S_T as coming from the same distribution and infer from the labelled source data to the target data.

But the landmarks are not in the same order, assuming there is a related order between the landmarks of each distribution, so the landmark spaces could be not coincident . If we take the trivial situation where both $\mathcal{D}_S$ and $\mathcal{D}_T$ are the same, then we can expect the GMMs to be able to place their clusters in the same way. But the randomness of the process could results in the source and target clusters not being provided in the same order by the different GMMs, so **LAMBDA** has to reorder the source or target landmarks to align them and to make the landmark spaces coincide .

To address this issue, **LAMBDA1** tries all possible alignments between the landmarks of the different distributions to find the optimal one, comparing two alignments using a defined cost function. This function takes an alignment, *i.e.* a specific order for the target landmarks, and returns its cost, relative to the dissimilarity between the ordered source and target landmarks. The proposed cost function is composed of two different sub-costs functions, which are specific costs based on some aspects of dissimilarity between aligned landmarks:

$$h_1(\boldsymbol{\mu}^S, \boldsymbol{\mu}^T, \boldsymbol{\sigma}) = \sum_{k=1}^{K}\sum_{k'=1}^{K}\Big(\|\boldsymbol{\mu}_k^S - \boldsymbol{\mu}_{k'}^S\|_2 - \|\boldsymbol{\mu}_{\boldsymbol{\sigma}(k)}^T - \boldsymbol{\mu}_{\boldsymbol{\sigma}(k')}^T\|_2\Big)^2$$

$$h_2(\mathbf{p}^S, \mathbf{p}^T, \boldsymbol{\sigma}) = \sum_{k=1}^{K}\Big(p_k^S - p_{\boldsymbol{\sigma}(k)}^T\Big)^2$$

where $\{\boldsymbol{\mu}_k\}_{k=1}^K$ are the landmarks of each distribution, $\{p_k\}_{k=1}^K$ are the proportions of the gaussian clusters from which the landmarks are derived on both source and target distribution, and $\boldsymbol{\sigma}(k)$ is the permutation index of the k-th target landmark.
The final cost function is then a linear combination of these two sub-costs:

$$\mathbf{c}(\boldsymbol{\mu}^S, \boldsymbol{\mu}^T, \mathbf{p}^S, \mathbf{p}^T, \boldsymbol{\sigma}) = \lambda h_1(\boldsymbol{\mu}^S, \boldsymbol{\mu}^T, \boldsymbol{\sigma}) + (1-\lambda)h_2(\mathbf{p}^S, \mathbf{p}^T, \boldsymbol{\sigma}),$$

where λ is a hyper-parameter of **LAMBDA**, which defines the balancing of the importance of the two costs. Thus, the pairing problem can be seen as finding the best permutation $\boldsymbol{\sigma}$ solution of the following optimization problem:

$$\arg\min_{\boldsymbol{\sigma}} \mathbf{c}(\boldsymbol{\mu}^S, \boldsymbol{\mu}^T, \mathbf{p}^S, \mathbf{p}^T, \boldsymbol{\sigma}).$$

Once **LAMBDA** has the optimal landmarks alignment, it simply projects the datapoints expressing them with the distances to the ordered landmarks of their respective distributions. Then, the source and target data are in the landmark spaces, which are expected to coincide , and a classical classifier can learn on the projected source data in order to perform the inference on the projected target data.

Algorithm 1 shows all the steps of the **LAMBDA1** algorithm. $\boldsymbol{\mu}_S$ and $\boldsymbol{\mu}_T$ will then be kept for the prediction function to be able to project new data into the same landmark spaces. **LAMBDA** is then able to predict on data coming from either distribution (given the information of which one the new data is coming from).

Algorithm 1: LAMBDA1

Input: $S_S = \{(\mathbf{x}_i^s, y_i)\}_{i=1}^{m_s}$: Source training set.
$S_T = \{(\mathbf{x}_i^t)\}_{i=1}^{m_t}$: Target training set.

begin
- $GMM_S \leftarrow Select\_best\_GMM(S_S, K, ntry)$
- $\boldsymbol{\mu}^S \leftarrow GMM_S.means$
- $\mathbf{p}^S \leftarrow GMM_S.proportions$
- $GMM_T \leftarrow Select\_best\_GMM(S_T, K, ntry)$
- $\boldsymbol{\mu}^T \leftarrow GMM_T.means$
- $\mathbf{p}^T \leftarrow GMM_T.proportions$
- $best\_cost \leftarrow +\infty$
- $best\_\boldsymbol{\sigma} \leftarrow NULL$
- **foreach** $\boldsymbol{\sigma}$ *in get_permutations*$(1..K)$ **do**
  - $cost$ = $\mathbf{c}(\boldsymbol{\mu}^S, \boldsymbol{\mu}^T, \mathbf{p}^S, \mathbf{p}^T, \boldsymbol{\sigma})$
  - **if** $cost < best\_cost$ **then**
    - $best\_cost \leftarrow cost$
    - $best\_\boldsymbol{\sigma} \leftarrow \boldsymbol{\sigma}$
- $\mathbf{x}_i'^s \in \mathbb{R}^K \leftarrow \{\|\mathbf{x}_i^s - \boldsymbol{\mu}_k^S\|_2\}_{k=1}^K, \forall i \in [\![1, m_s]\!]$

As shown in the experimental results in Section 4, **LAMBDA1** is very strong, even invariant, to rotations and translations, because the landmarks allows the source and target distributions to be projected independently, and the GMMs allows the landmarks to also be placed independently between the distributions. And if the distances are not preserved by a scaling transformation from the source to the target domain, **LAMBDA1** can first normalize the data, setting it at approximately the same scale, in order to simplify the transformation the model has to overcome, and making the alignments comparisons more reliable. But this normalization should not always be done because it increases the variance of the results, giving slightly worse loss on average (except for the scaling transformations).
Thus, it does not matter to **LAMBDA1** at what distance, angle or different scaling the distributions are from each other, and two different classes coming from different distributions can even overlap: the model will still be able to correctly project them in coincident landmarks spaces.

However, the alignment step is very heavy to compute, as

LAMBDA1 must try each of the possible alignments between the source and target landmarks, resulting in an algorithm of factorial complexity of K because of the number of possible permutations of the target landmarks.

3.2 A Refined Version

We could express the problem of finding the optimal alignment as a (balanced) assignment problem, since this problem consists in finding the least costly way of assigning tasks to workers, the balanced aspect corresponding to the fact that there are as many workers as tasks, and since **LAMBDA** tries to assign each target landmark to a source landmark in a way that minimises a cost.

The only problem with this formulation is that it needs a local-cost function, which gives a cost for a pair of landmarks. But we defined previously a global-cost function, which gives the cost of a whole alignment, and this local cost is difficult to define without risking a bias in the evaluation of an alignment. Indeed, in our case the cost between two landmarks depends on the entire landmarks sets, and more precisely on the distances between a landmark and the other ordered ones of its distribution.

On the other hand, there are algorithms that solve the assignment problem much more efficiently than going through all the possible permutations (having a has a factorial complexity of K) , with a polynomial complexity of the number of tasks/workers, *i.e.* K in our case.

Hungarian Algorithm. The Hungarian algorithm [17] is one of the first algorithms, and therefore one of the best known, to solve the assignment problem in polynomial time. It works by performing operations on a given cost matrix, in which each line corresponds to a worker, each column corresponds to a task, and the value in a cell corresponds to the cost of assigning the corresponding task to the corresponding worker.

The idea of this algorithm is to decrease the values of the cost matrix until we have some zeros, then play with heuristics [20] to move the zeros until we have a set of zeros corresponding to an optimal assignment, a zero in the i-th row and j-th column corresponding to the assignment of the j-th task to the i-th worker.

This algorithm is perfect to assign each target landmark to a source landmark, as long as we are able to give it a $\boldsymbol{\mu}^S$-to-$\boldsymbol{\mu}^T$ local-cost matrix.

Landmarks alignment of LAMBDA2. In order to be able to calculate a cost matrix to include the Hungarian algorithm in **LAMBDA**, we have defined a local-cost function which takes two landmarks and returns the cost of aligning them, relative to their dissimilarities, especially with respect to their distances to other landmarks in their proper distributions. The local-cost function is also composed of two sub-costs functions, derived from the previous h_1 and h_2:

$$h'_1(\boldsymbol{\mu}_i^S, \boldsymbol{\mu}_j^T, \bar{\boldsymbol{\mu}}^S, \bar{\boldsymbol{\mu}}^T) = \sum_{k=1}^{K} \left(\|\boldsymbol{\mu}_i^S - \bar{\boldsymbol{\mu}}_k^S\|_2 - \|\boldsymbol{\mu}_j^T - \bar{\boldsymbol{\mu}}_k^T\|_2 \right)^2$$

$$h'_2(p_i^S, p_j^T) = \left(p_i^S - p_j^T\right)^2$$

where $\bar{\boldsymbol{\mu}}^S$ and $\bar{\boldsymbol{\mu}}^T$ are the landmarks sorted from nearest to farthest from $\boldsymbol{\mu}_i^S$ and $\boldsymbol{\mu}_j^T$ respectively.
The final local-cost function is then a linear combination of these two sub-cost functions, using the same hyper-parameter as **LAMBDA1**:

$$\mathbf{c}'(\boldsymbol{\mu}_i^S, \boldsymbol{\mu}_j^T, \bar{\boldsymbol{\mu}}^S, \bar{\boldsymbol{\mu}}^T, p_i^S, p_j^T) = \lambda h'_1(\boldsymbol{\mu}_i^S, \boldsymbol{\mu}_j^T, \bar{\boldsymbol{\mu}}^S, \bar{\boldsymbol{\mu}}^T) + (1-\lambda) h'_2(p_i^S, p_j^T)$$

LAMBDA2 computes the two distance matrices between the landmarks of each distribution before constructing the local-cost matrix from $\mathbf{c}'$, to avoid having to compute it every time. Then, each i-th row of the matrices is sorted so that it corresponds to $\bar{\boldsymbol{\mu}}$ of $\boldsymbol{\mu}_i$.

Finally, except for this part of landmarks alignment with the optimal alignment search algorithm and its local-cost function, **LAMBDA2** works exactly like **LAMBDA1**.

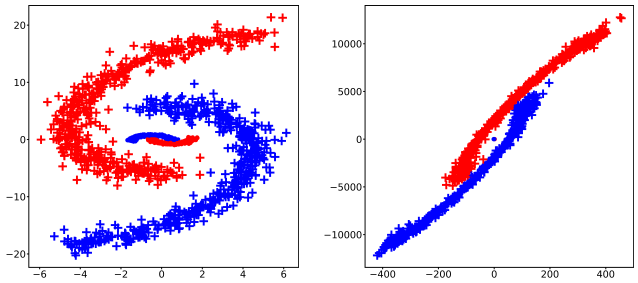
4 Experimental Results

In our experiments, we tested **LAMBDA1** and **LAMBDA2**, both with and without a normalisation (named "**LAMBDA<i> (n)**" in the following results table), to compare the ability of these 4 versions to handle the different transformations, and to confirm the hypotheses behind the mechanisms of the model.

4.1 Settings

Dataset. We chose the moons dataset as the basis of our tests because, even with basic linear transformations it is quite easy to come up with interesting special cases that quickly become potentially hard to overcome for the other models, but not for **LAMBDA**.
Another argument that makes the moons very interesting to evaluate on the different versions of **LAMBDA** is the symmetry of its structure and classes. Indeed, since **LAMBDA1** has a global cost function as heuristic to search for the best alignment, it takes into account all the landmarks according to each other. On the other hand, **LAMBDA2** has a local cost function as heuristic, which makes it take into account only locally the landmarks, and so it is likely that it exchanges symmetric landmarks as it will see them as similar. This explains the greater variation in the performances of **LAMBDA2** on this type of data compared to **LAMBDA1**.

However, to prevent an easy estimation of the density for the models who can do it in a simple manner, we made a more complex noise by generating several layers of moons with different noise levels.



(a) simple transformation matrix

(b) complex transformation matrix

Figure 3: Matrix transformations

And to ensure that we have captured as much as possible of the nuances of each model's performance and capabilities, like overcoming some special cases arising from the transformations, we first used the 3 basics linear transformations, namely (i) the rotation, (ii) the translation and (iii) the scaling to generate the target from the source distribution. Thus, 5 test cases were made out of each, to shift the two distributions apart gradually and see when and how each model will become bad at analysing them.
For the sake of reproducibility of our experiments: the rotations are made clockwise around the mean, the translations are expressed by the translated distance of each axis (x, y), and the scalings are done in a way that preserves the mean of the data.
To further compare and separate the models that seems invariant to linear transformations, we made a few more difficult test cases: one in which the classes significantly *overlap* between the distributions, and another in which there is a *mirror* transformation (or negative scaling) on the x axis. We also tested linear transformations based on matrices between the source and the target, where $\mathbf{x}^T = \mathbf{x}^T \cdot \mathbf{M}$. A so-called simple transformation and a second one called complex whose transformations are respectively given by:

$$\begin{pmatrix} 2 & 12 \\ 7 & 1 \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} 243 & 7256 \\ 92 & 47 \end{pmatrix}.$$

These matrices transformations seems to be the limitation of our set of models, since even the performances of **LAMBDA** start to decrease with them. It shows, at two different levels, hard transformations to overcome, as we can see in Figure 3. The result of the simple matrix transformation (Figure 3a) is close to a rotation together with a scaling (higher on y than on x), where the result of the complex matrix transformation (Figure 3b) is completely distorted and strongly stretched diagonally.

Competitors. We compared the 4 versions of **LAMBDA** to some state-of-the-art non-deep models, whose hyper-parameters are tuned using a the reverse cross validation (RCV) procedure [12, 28]. *This consists of using the predicted labels on the target data to know if a model learned with these labels is able to retrieve the labels of the source data.*

- **DASVM** [2]. This domain adaptation algorithm proposes to iteratively adapt the decision boundary by progressively including a number $\rho = \lceil \frac{m_t}{v} \rceil$, with $v \in \{20, 50, 100\}$ of target data labeled at the previous iteration. More, precisely, it will include the target labeled data that close to margin of the learned SVM. The iteratively learned classifier then depends on two parameters $C \in 10^{\{-2,\dots,2\}}$ (associated to the risks on the source data, and eventually $\gamma \in \frac{2^{\{-2,\dots,2\}}}{d}$ if we use a gaussian kernel), which is tuned on the source data, and $C^* = \frac{\tau C}{v}$, with $v \in \{1, 5, 10\}$ and $\tau = 0.5$, associated to a risk measure on the target data. The maximum number of iterations γ' for **DASVM** is tuned in $\{2, 3, 5\}$.
  Note that the parameters v, γ' and $C^\star$ are tuned via the RCV, while C and γ are tuned on the source data.

- **PBDA** [11]. It aims to learn a representation of the data using a kernel. The parameters of this kernel are optimized such that it minimizes the empirical risk on the source samples, controlled by a hyper-parameter $C \in 10^{\{-2,\dots,2\}}$ and the domain disagreement controlled by a hyper-parameter $A \in 10^{\{-2,\dots,2\}}$.

- **DALC** [13]. It works in a similar manner than **PBDA**. However, the quantities that are involved in the minimization of the generalization error on the source risk are different. It depends on the target disagreement, weighted by a hyper-parameter $C \in 10^{\{-2,\dots,2\}}$ and the source joint error weighted by a hyper-parameter $B \in 10^{\{-2,\dots,2\}}$. The linear classifier is then learned based on a gaussian kernel depending on a hyper-parameter $\gamma \in \{2, 3, 5\}$.

- **LSSA** [1]. This method consists in (i) selecting landmarks among the source and target examples. We retain the points of these sets for which the overlap of the distribution of the source and target Gaussian similarities is higher than a certain $threshold \in \{0.2, 0.3, 0.5, 0.75, 0.9\}$. (ii) These landmarks are then used to obtain a new representation of the data using a Gaussian similarity depending on a parameter $\sigma \in \{0.1, 0.5, 1, 2, median(distances)\}$, where $median(distances)$ is the default value recommended by the authors, and corresponds to the median distance between any pair of points drawn randomly from source and target data. This representation is then reduced (iii) using a PCA in a space of dimension $d' \in \{5, 10, 20, 50, 100\}$ for the source and target data respectively. Finally, (iv) a linear transformation is then learned to align the two representations.

We choose to compare ourselves to approaches based on the reduction of the distance between the two domains [11, 13] in order to show that this approach, although theoretically founded, does not allow to treat simple cases in practice.
We also compare ourselves to [1] in order to show that the landmark selection method as well as the proposed pairing

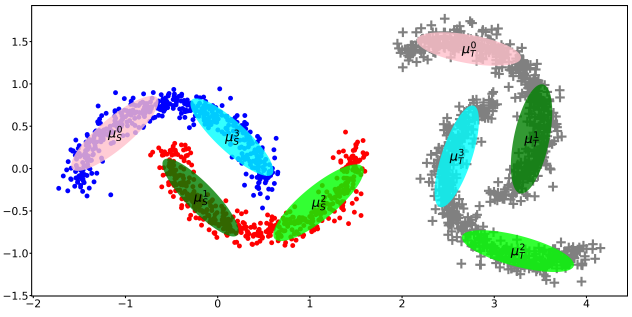


Figure 4: A wrong alignment caused by too few clusters. We can see that on source data (dots), μ_S^0 and μ_S^3 are on the same moon, where on the target data (plus), their pairs μ_T^0 and μ_T^3 are on different moons.

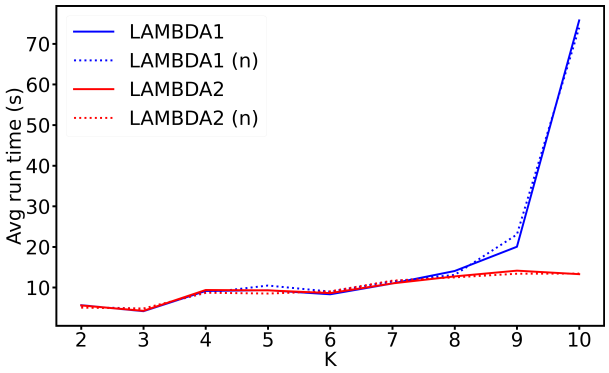


Figure 5: The average run time of 5 **LAMBDA** learning processes for each K.

method allow, in a similar time to solve these tasks as well or even more efficiently with a reduced number of landmarks.

Furthermore, to have an absolute baseline, we also compared them to a simple classifier, namely SVM. To align with these models, we also chose an SVM as the internal classifier of both **LSSA** and **LAMBDA**. As the SVM is a simple classifier, and in this work the focus is on the domain adaptation performance of the models, we did not fine-tuned the SVM, neither as an internal classifier nor as baseline. So the default parameter values have been kept, namely the rbf (gaussian) kernel is used, $C = 1$, and $\gamma = \frac{1}{d \cdot var(\mathbf{X}_S)}$, where $var(\mathbf{X}_S)$ is the total variance of the source data[2].

DALC, **PBDA** and **LSSA** have been fine-tuned with grid-searches, through an RCV process with 5 folds, on their main hyper-parameters.

Unlike them, **DASVM** has not been fine-tuned by a grid search of our own, as a fine-tuning process is already included directly in the fit function of the available authors implementation.

However, **LAMBDA** has not been fine-tuned at all because its inner workings are inherently unsuitable for RCV. Indeed, the performances of the model essentially depend on the way the GMMs place their clusters, whatever the value of K. The hyper-parameter λ can decide if the model correctly interprets the clusters, but if the clusters are badly placed, there is no way to ensure good predictions of the model since there is probably no good alignment.

That said, the RCV works by trying all the given values of the hyper-parameters, in this case K and λ, and will compare the results of the reversed predictions of the model. But if a good K and/or a good λ are tested in pair with clusters that are not placed well, **LAMBDA** will inevitably have bad results, which will probably prevent the parameters values from being selected by the grid-search algorithm.

We then arbitrarily fixed the value of λ for all our experiments to $\frac{2}{3}$, since when λ tends to 1 it favours analysis on the distances between cluster means, when it tends to 0 it favours analysis on the proportions of clusters (*c.f.* formulas of c and c' in Section 3), and given our data, our intuition is to give more importance to the distance between landmarks rather than their respective proportions in the GMM.

On the other hand, to fix the value of K, we ran **LAMBDA** instances on several samples of data with K going from 4 to 10, to see the effects on the clusters and on the alignment. We noticed that too few clusters causes the model to often get mixed up with them, like in Figure 4 where **LAMBDA** exchanged two opposite clusters in the alignment. On the contrary, having too many clusters painfully slows down the algorithm, at least for **LAMBDA1**, as we can see in Figure 5.

Thus, $K = 6$ was set for all experiments, as a balance between the two constraints.

For each test case or RCV run, a total number of 3000 datapoints is randomly generated, split 67% and 33% for source and target data respectively. Then, for the RCV process, the source distribution is itself split into 5 folds, and for the evaluation runs of a test case, the target distribution is split into 67% and 33% for the training and test data.

The error rate of the 10 runs of each test case were then averaged, keeping the standard deviation for visualising the uncertainty. Both are reported in Table 1 and analysed in the following section.

4.2 Results on Synthetic Data

The Table 1 shows the performances of all the models, expressed as the mean of the 10 runs with the standard deviation in brackets. The *(n)* after **LAMBDA** indicates that the model has normalised the data before placing and aligning the landmarks.

PBDA, **DALC** and **LSSA** have been fine-tuned with a grid-search for each test case, on their hyper-parameters listed in the previous section.

On the other hand, **DASVM** has an internal fine-tuning process which has fine-tuned its parameters. In addition, SVM and **LAMBDA** have not been fine-tuned at all.

LAMBDA versions. About the different versions of **LAMBDA**, Table 1 shows that **LAMBDA2** handles the

[2]Default parameters of the function svm.svc in sklearn

		SVM	PBDA	DALC	DASVM	LSSA	LAMBDA1	LAMBDA1 (n)	LAMBDA2	LAMBDA2 (n)
avg running time		0.08s	32.13s	18.63s	1.03min	6.1s	9.16s	8.66s	9.18s	7.97s
Rotations	20°	0.015 (0.006)	0.015 (0.014)	0.003 (0.004)	0.029 (0.008)	0.021 (0.015)	**0.001** (0.001)	0.004 (0.004)	0.04 (0.116)	0.056 (0.119)
	40°	0.055 (0.01)	0.252 (0.064)	0.039 (0.011)	0.076 (0.015)	0.065 (0.035)	**0.002** (0.002)	0.004 (0.006)	0.061 (0.118)	0.042 (0.121)
	60°	0.105 (0.013)	0.339 (0.022)	0.129 (0.024)	0.131 (0.015)	0.22 (0.058)	**0.002** (0.004)	0.048 (0.139)	0.07 (0.143)	0.002 (0.003)
	90°	0.178 (0.02)	0.457 (0.013)	0.159 (0.019)	0.211 (0.017)	0.457 (0.041)	0.002 (0.003)	**0.001** (0.001)	0.07 (0.131)	0.081 (0.159)
	120°	0.291 (0.022)	0.281 (0.035)	0.392 (0.047)	0.321 (0.023)	0.594 (0.057)	0.004 (0.007)	0.023 (0.058)	0.033 (0.089)	**0.003** (0.005)
Translations	(0, 0.25)	0.032 (0.01)	0.038 (0.012)	0.027 (0.016)	0.034 (0.007)	0.009 (0.009)	**0.002** (0.002)	0.135 (0.206)	0.163 (0.162)	0.05 (0.148)
	(0, 1)	0.36 (0.026)	0.236 (0.038)	0.334 (0.015)	0.381 (0.046)	0.004 (0.003)	**0.003** (0.004)	0.05 (0.144)	0.097 (0.147)	0.092 (0.178)
	(0, 10)	0.508 (0.025)	0.485 (0.039)	0.507 (0.084)	0.492 (0.026)	0.01 (0.004)	**0.009** (0.021)	0.091 (0.178)	0.064 (0.122)	0.057 (0.118)
	(0.25, 1)	0.309 (0.022)	0.318 (0.026)	0.307 (0.021)	0.317 (0.027)	0.008 (0.005)	**0.001** (0.001)	0.002 (0.003)	**0.001** (0.002)	0.015 (0.037)
	(1, 10)	0.508 (0.025)	0.494 (0.023)	0.581 (0.293)	0.492 (0.026)	0.005 (0.007)	0.003 (0.004)	**0.001** (0.002)	0.076 (0.153)	0.175 (0.213)
Scalings	0.1	0.519 (0.024)	0.612 (0.03)	0.569 (0.039)	0.496 (0.027)	**0.006** (0.004)	0.354 (0.065)	0.049 (0.137)	0.369 (0.109)	0.095 (0.186)
	0.5	0.451 (0.021)	0.16 (0.01)	0.486 (0.04)	0.38 (0.07)	0.004 (0.005)	0.373 (0.051)	**0.003** (0.003)	0.385 (0.046)	0.061 (0.131)
	1.25	0.005 (0.004)	0.004 (0.004)	0.016 (0.022)	0.016 (0.006)	0.009 (0.005)	0.03 (0.021)	**0.002** (0.003)	0.038 (0.069)	0.09 (0.175)
	2	0.061 (0.009)	0.073 (0.011)	0.057 (0.012)	0.071 (0.016)	**0.003** (0.002)	0.359 (0.047)	**0.003** (0.005)	0.386 (0.069)	0.083 (0.131)
	10	0.514 (0.026)	0.517 (0.051)	0.77 (0.066)	0.28 (0.047)	0.026 (0.037)	0.474 (0.017)	**0.002** (0.003)	0.478 (0.014)	**0.002** (0.004)
overlapping		0.606 (0.129)	0.715 (0.013)	0.613 (0.038)	0.84 (0.119)	0.013 (0.012)	**0.002** (0.003)	0.193 (0.02)	0.03 (0.087)	0.3 (0.137)
mirror		0.476 (0.032)	0.471 (0.021)	0.525 (0.029)	0.37 (0.013)	0.729 (0.017)	**0.002** (0.004)	0.026 (0.066)	0.092 (0.138)	0.073 (0.111)
matrix (simple)		0.507 (0.19)	0.554 (0.037)	0.985 (0.026)	0.492 (0.083)	0.441 (0.014)	0.469 (0.018)	**0.004** (0.002)	0.481 (0.015)	0.056 (0.1)
matrix (complex)		0.503 (0.017)	0.536 (0.036)	0.505 (0.027)	0.466 (0.004)	0.449 (0.034)	0.485 (0.012)	**0.24** (0.041)	0.483 (0.015)	0.306 (0.057)

Table 1: Average (and standard deviation) of 10 executions of test cases, consisting of the moons dataset with the basic linear transformations and a few complex ones between source and target data. Our model **LAMBDA** is confronted with state-of-the-art **PBDA**, **DALC** and **LSSA** which have been fine-tuned with a grid-search for each test case, as well as **DASVM** which has its own internal grid-search, and **SVM**. The average running time only concerns the evaluation, not the grid-search running time since not all models perform one.

transformations almost the same way as **LAMBDA1**, except that it often has more variance in its results.

We occasionally find some error rates higher than its surroundings, because sometimes the GMMs of the model misplaces the clusters, which affects very much its performances, making the mean and standard deviation increase. For example, **LAMBDA1** (n) has worse results on 60° rotation (0.048) but performs well on 40° and 90° rotations (0.004 and 0.001 respectively). And that is because for the 60° rotation the clusters has been misplaced and/or misaligned. But if the test cases are run again, their results could be slightly different.

However, we can see that with the normalisation, this phenomenon increases. We believe that the normalisation brings the different distances between the dense areas closer together, confusing the GMMs a bit in their cluster placements.

In addition, **LAMBDA2** also has a higher variance in its results, and does not generally provide better results than **LAMBDA1**. That is because its local-cost function is an approximation of the global-cost function of **LAMBDA1**, to speed up the computation with high K, as shown in Figure 5.

This approximation sometimes confuses the model in the alignment step, making it exchanging some clusters because they are locally similar, like in Figure 6.

Anyway, **LAMBDA** without normalisation handles very well rotations and translations, and almost seems to be invariant to them, with an error rate below 0.005 most of the time. We can attribute this to the facts that the distributions are treated independently by the model and that the projection is made according to the landmarks which are placed according to the datapoints. So the model can handle each distribution wherever it is located. Moreover, the projection is made with the distances to the landmarks, so it is not dependent to the rotation of the distribution.

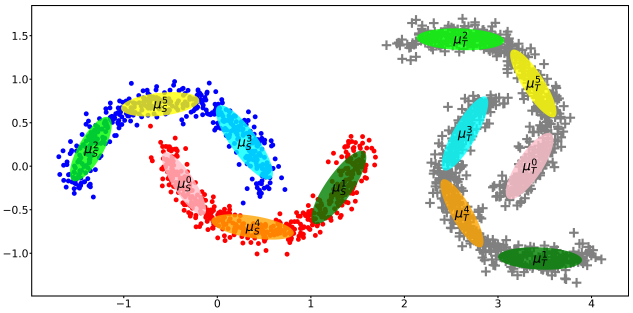


Figure 6: Wrong alignment due to **LAMBDA2**'s approximation of the local-cost function. $\boldsymbol{\mu}^0$ (pink) and $\boldsymbol{\mu}^3$ (blue) landmarks are exchanged between source and target distributions. The landmarks appear locally similar, as they are roughly in the centre of their distributions and almost equidistant from the three landmarks on the opposite moon. Symmetry of the data leads to symmetry of the landmarks.

In contrast, it seems to handle low-amplitude scalings only, and the stronger the scaling, the higher the loss rate. Indeed, a scaling modifies significantly the distances computed during the projections. But that is why we included the optional normalisation of the data on the model, and it seems to be very effective as **LAMBDA** remains in the lead for all test cases with its 4 versions, except for the 0.1 scaling. For this particular test case, we believe that the versions with the normalisation had a misplacement of its clusters

(plus the variance of **LAMBDA2**), like for the 60° rotation of **LAMBDA1 (n)**, which makes **LAMBDA** let **LSSA** take the lead. Indeed, it does not matter what distribution is the source of the target one, in the placement of the clusters, or in their alignment, and a 0.1 scaling or a 10 scaling is basically the same.

However, the results of **LAMBDA** are still relatively interesting in general, and most of the time as good or better than the other models. It seems that its landmarks spaces coincidence allows its internal classifier to perform well as if it was on the same distribution with just a little noise.

But we can see that we have reach the limits of the model with the complex matrix, at least with these fixed values of its hyper-parameters.

Comparisons with other models. In contrast to **LAMBDA**, the other models seem to handle low-amplitude transformations only, and do not seem to be able to handle too important transformations. Except for **LSSA**, which seems to handle translations and scalings almost as well as **LAMBDA**, but it is not so good on strong rotations.

For example, the 20° rotation is handled by all other models with an error rate of at most 0.029. But when we increase the rotation, they are unable to keep these performances, like the 60° rotation and their error rates between 0.09 and 0.2.

While the 20° and 60° rotations are handled approximately the same way by **LAMBDA**. And the same can be seen with translations and scalings, except for **LSSA** which seems to be able to handle them almost as good as **LAMBDA**, and even takes the lead for some scalings.

In fact, **LSSA** is a good model, since it runs quite quickly, and keeps good results on most of the tested transformations. It handles translations and scalings particularly well, thanks to its normalisations of the data, and, as **LAMBDA**, processes both distributions independently.

But it does not seem to be able to handle wide rotations, and seems totally lost when dealing with a target distribution that is a mirror image of the source distribution.

This could be explained by the fact that **LSSA** uses PCAs, which by definition lose information, or by the fact that it uses the same landmarks for both source and target data. Indeed, even if it reduces this second effect with independent PCAs on each distribution, a landmark feature can be kept inside both PCA reduction. And if a landmark feature is in common on the final source and target representations, but with a rotation between the original distributions, the SVM may try to interpret this feature in a more or less similar way. And **LAMBDA** can totally handle these cases quite accurately.

5 Conclusion

LAMBDA is quite efficient at overcoming geometric transformations and compares very well to the state-of-the-art models, namely **PBDA**, **DALC**, **DASVM** and **LSSA**. Indeed, it seems invariant to the basic transformations and roughly robust to matrix transformations, compared to the other models.

In addition, the way it handles distributions independently allows it to handle different complex situations that, to our knowledge, cannot be all solved by one existing model. For example, **LAMBDA** can distinguish two different classes that overlap between $\mathcal{D}_S$ and $\mathcal{D}_T$. It can also deal with a non-binary classification task, since it is totally unsupervised in its alignment. It simply delegates the labels consideration to the classification task, which therefore depends entirely on the ability of the internal classifier to handle the problem.

In addition, **LAMBDA** generalises very well on more than two distributions simultaneously, by having a set of K landmarks for each one and an alignment search on all of these sets.

Furthermore, **LAMBDA** can easily handle source and target distributions with different number of dimensions, since the GMMs and the projections into the landmark spaces are done independently on the source and target domains. Thus, our method can also be used for Heterogeneous Domain Adaptation, in which the number of dimensions in the source and target data is different. To address this, **LAMBDA** can simply place the same number of landmarks on each distribution, and is able to compare them, like with the actual cost function which compares them through intra-distribution distances and proportions.

Thus, there is some work to be done to improve the model since it is very sensitive to the quality of the placement of the landmarks by the GMMs, without which the landmarks are less likely and therefore their variance prevents their alignment.

In addition, we are not yet able to perform a grid-search on **LAMBDA**, due to this random variance. In this paper we have fixed the hyper-parameters of the model, and it is only possible because we can visualise the data. On high dimensional data, and/or with much more complex structures, it will be difficult to do the same.

To overcome this issue, we could imagine a metric which compares the estimations of the source and target GMMs, and indicates if the clusters distributions are close enough for the alignment step. This kind of metric exists in clustering for example [9], but is used to compare different partitions on the same data, where in our case we have similar but different data.

Furthermore, GMMs are known to be time and memory consuming when dealing with high dimensional datasets. Some other statistical algorithm or method for placing landmarks could be more appropriate, like a PCA for a preprocess, at the risk of loosing information.

References

[1] Rahaf Aljundi, Rémi Emonet, Damien Muselet, and Marc Sebban. Landmarks-based kernelized subspace alignment for unsupervised domain adaptation. In

Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 56–63, 2015.

[2] Lorenzo Bruzzone and Mattia Marconcini. Domain adaptation problems: A dasvm classification technique and a circular validation strategy. *IEEE transactions on pattern analysis and machine intelligence*, 32(5):770–787, 2009.

[3] Gilles Celeux and Gérard Govaert. A classification em algorithm for clustering and two stochastic versions. *Computational statistics & Data analysis*, 14(3):315–332, 1992.

[4] Gilles Celeux and Gérard Govaert. Gaussian parsimonious clustering models. *Pattern recognition*, 28(5):781–793, 1995.

[5] Chenhui Chu and Rui Wang. A survey of domain adaptation for neural machine translation. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 1304–1319, Santa Fe, New Mexico, USA, August 2018. Association for Computational Linguistics.

[6] Abolfazl Farahani, Sahar Voghoei, Khaled Rasheed, and Hamid R Arabnia. A brief review of domain adaptation. *Advances in Data Science and Information Engineering: Proceedings from ICDATA 2020 and IKE 2020*, pages 877–894, 2021.

[7] Basura Fernando, Amaury Habrard, Marc Sebban, and Tinne Tuytelaars. Unsupervised visual domain adaptation using subspace alignment. In *Proceedings of the IEEE international conference on computer vision*, pages 2960–2967, 2013.

[8] Mario A. T. Figueiredo and Anil K. Jain. Unsupervised learning of finite mixture models. *IEEE Transactions on pattern analysis and machine intelligence*, 24(3):381–396, 2002.

[9] Guojun Gan, Chaoqun Ma, and Jianhong Wu. Evaluation of clustering algorithms. In *Data Clustering: Theory, Algorithms, and Applications, Second Edition*, chapter 18, pages 277–295. SIAM, 2020.

[10] Bo Geng, Dacheng Tao, and Chao Xu. Daml: Domain adaptation metric learning. *IEEE Transactions on Image Processing*, 20(10):2980–2989, 2011.

[11] Pascal Germain, Amaury Habrard, François Laviolette, and Emilie Morvant. A pac-bayesian approach for domain adaptation with specialization to linear classifiers. In *International conference on machine learning*, pages 738–746. PMLR, 2013.

[12] Pascal Germain, Amaury Habrard, François Laviolette, and Emilie Morvant. Pac-bayes and domain adaptation. *Neurocomputing*, 379:379–397, 2020.

[13] Pascal Germain, François Laviolette, Amaury Habrard, and Emilie Morvant. A new pac-bayesian view of domain adaptation. In *NIPS 2015 Workshop on Transfer and Multi-Task Learning: Trends and New Perspectives*, 2015.

[14] Arthur Gretton, Karsten M Borgwardt, Malte J Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *The Journal of Machine Learning Research*, 13(1):723–773, 2012.

[15] Jiayuan Huang, Arthur Gretton, Karsten Borgwardt, Bernhard Schölkopf, and Alex Smola. Correcting sample selection bias by unlabeled data. *Advances in neural information processing systems*, 19, 2006.

[16] Morvarid Karimpour, Shiva Noori Saray, Jafar Tahmoresnezhad, and Mohammad Pourmahmood Aghababa. Multi-source domain adaptation for image classification. *Machine Vision and Applications*, 31:1–19, 2020.

[17] Harold W Kuhn. The hungarian method for the assignment problem. *Naval research logistics quarterly*, 2(1-2):83–97, 1955.

[18] Zhihe Lu, Yongxin Yang, Xiatian Zhu, Cong Liu, Yi-Zhe Song, and Tao Xiang. Stochastic classifiers for unsupervised domain adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9111–9120, 2020.

[19] Liang Mi, Wen Zhang, and Yalin Wang. Regularized wasserstein means for aligning distributional data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 5166–5173, 2020.

[20] James Munkres. Algorithms for the assignment and transportation problems. *Journal of the society for industrial and applied mathematics*, 5(1):32–38, 1957.

[21] Alan Ramponi and Barbara Plank. Neural unsupervised domain adaptation in nlp—a survey. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6838–6855, 2020.

[22] Ievgen Redko, Emilie Morvant, Amaury Habrard, Marc Sebban, and Younes Bennani. *Advances in domain adaptation theory*. Elsevier, 2019.

[23] Gideon Schwarz. Estimating the dimension of a model. *The annals of statistics*, pages 461–464, 1978.

[24] Baochen Sun, Jiashi Feng, and Kate Saenko. Correlation alignment for unsupervised domain adaptation. *Domain adaptation in computer vision applications*, pages 153–171, 2017.

[25] Remi Tachet des Combes, Han Zhao, Yu-Xiang Wang, and Geoffrey J Gordon. Domain adaptation with conditional distribution matching and generalized label shift. *Advances in Neural Information Processing Systems*, 33:19276–19289, 2020.

[26] Alessio Tonioni, Matteo Poggi, Stefano Mattoccia, and Luigi Di Stefano. Unsupervised domain adaptation for depth prediction from images. *IEEE transactions on pattern analysis and machine intelligence*, 42(10):2396–2409, 2019.

[27] Yueming Yin, Zhen Yang, Haifeng Hu, and Xiaofu Wu. Metric-learning-assisted domain adaptation. *Neurocomputing*, 454:268–279, 2021.

[28] Erheng Zhong, Wei Fan, Qiang Yang, Olivier Verscheure, and Jiangtao Ren. Cross validation framework to choose amongst models and datasets for

transfer learning. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2010, Barcelona, Spain, September 20-24, 2010, Proceedings, Part III 21*, pages 547–562. Springer, 2010.